

Proyecto programación orientada a objetos

Estudiantes:

Samuel Marroquín
María Camila Pradilla
John Fredy Gallego Zambrano

Ingeniería de Sistemas / Computación Científica

Proyecto:

Core-Dump: System Recovery

Docente:

Jesús Andrés Hincapié Londoño

Universidad de Medellin

2026-1

Descripción del problema

Introducción:

El proyecto consiste en el desarrollo de un videojuego de aventura sistémica. El entorno de juego simula el interior de un sistema computacional donde las entidades lógicas han perdido su integridad y el usuario debe restaurarlas. La arquitectura y lógica interna estarán fundamentadas estrictamente en la Programación Orientada a Objetos (POO) utilizando Python. Para cumplir con los estándares del curso en la entrega final, el juego integrará una Interfaz Gráfica de Usuario (GUI) que facilitará la inmersión y la interacción del jugador con el entorno.

Reglas de Negocio y Mecánicas del Juego:

El núcleo del juego se rige por las siguientes mecánicas y reglas de interacción:

- **Entidades y Actores:** El ecosistema del juego contiene entidades con comportamiento activo, como el Jugador, personajes de apoyo (NPCs) y amenazas (Virus). Cada entidad posee estados propios (integridad, energía) que determinan su funcionamiento.
- **Exploración y Navegación:** El mundo virtual está segmentado en sectores o nodos del sistema, equivalentes a habitaciones o niveles. El progreso requiere que el jugador ejecute un sistema de movimiento entre estos sectores para descubrir áreas dañadas.
- **Gestión de Recursos e Inventario:** El entorno contiene objetos interactuables vitales para la misión, tales como Scripts, Parches y Llaves. La regla principal de progreso es la manipulación de estos objetos para restaurar el sistema. El jugador deberá administrarlos mediante un sistema de inventario.
- **Resolución de Conflictos:** Los encuentros con amenazas se manejan mediante un sistema de combate o resolución de conflictos. El éxito en estos encuentros se calcula matemáticamente basándose en los estados actuales del jugador y los recursos que decida utilizar en ese momento.

Objetivos Técnicos:

A nivel de desarrollo, el propósito principal es aplicar de forma práctica los cuatro pilares de la POO:

- **Abstracción:** Modelado de las entidades del mundo virtual en estructuras de datos lógicas.
- **Encapsulamiento:** Protección de los estados internos de los elementos del juego para evitar modificaciones no autorizadas.
- **Herencia:** Creación de jerarquías para optimizar la construcción de los distintos elementos del mundo.
- **Polimorfismo:** Implementación de mecánicas que permitan a distintos elementos del juego reaccionar de forma única ante una misma acción del jugador.

Alcance del Proyecto (MVP y Entrega Final):

- Sistema de movimiento entre nodos de la red.
- Sistema de resolución de conflictos.
- Inventario de objetos y manipulación de recursos.

Interfaz Gráfica de Usuario (GUI):

Aunque la lógica interactiva inicial partirá de la recepción y procesamiento de comandos del usuario, la entrega final implementará una interfaz gráfica. Esta reemplazará la consola de texto para brindar retroalimentación visual, clara y en tiempo real sobre el estado del jugador y el entorno.

Justificación Académica:

Este proyecto es altamente relevante para la formación en Computación Científica, ya que exige un diseño riguroso de la lógica de negocio. El desarrollo en Python permitirá demostrar un manejo avanzado de colecciones de datos y la lectura y procesamiento de información para la toma de decisiones dinámicas dentro del sistema.

1. Modelo de clases:

El siguiente diagrama refleja la arquitectura orientada a objetos implementada en la versión final del juego. Se incluyen todas las clases, sus atributos, métodos, relaciones de herencia, asociación, composición y dependencia, así como la jerarquía de excepciones personalizadas.

1.1 Identificación de entidades y características:

A partir del código final entregado, se identifican las siguientes clases con sus atributos:

Clase	Atributos	Archivo
Sector	nombre: str, dificultad: int	models.py
Item	nombre: str, potencia: int, precio: int	models.py
Actor (ABC)	integridad: int	models.py
Player	energia: int=50, bits: int=0, morral: list[Item]	models.py
Boss	escudo: int	models.py
Engine	jugador, sectores, jefe_final, nivel_actual, ejecutando, parche_pro	Engine.py
CoreDumpGUI	game, root, labels, canvases, frames, buttons	main.py
GameObject (ABC)	nombre: str	interfaces.py
Reportable (Protocol)	— (interfaz)	interfaces.py

1.2 Jerarquía de excepciones:

Se implementó una jerarquía de excepciones personalizadas que hereda de GameException, la cual a su vez hereda de la clase built-in Exception de Python:

Excepción	Hereda de	Se lanza cuando...
GameException	Exception	Base de todas las excepciones del juego
InsufficientEnergyError	GameException	Energía < 5 al intentar explorar
InsufficientBitsError	GameException	Bits insuficientes para comprar parche
EmptyInventoryError	GameException	Morral vacío al reparar o batallar
LowIntegrityError	GameException	Integridad ≤ 15 al intentar recargar
AllSectorsRepairedError	GameException	Todos los sectores ya fueron reparados

2. Requisitos funcionales:

Respecto a la versión inicial, se añadieron detalles de validación con excepciones, la lógica de la GUI y las condiciones de victoria/derrota.

ID	Nombre	Resumen
R1	Iniciar juego	Inicializar Engine, Player, Boss, Sector e Item con valores predefinidos y mostrar GUI
R2	Explorar sistema	Consumir energía; obtener bits aleatorios o recibir daño a la integridad
R3	Reparar sector	Usar un ítem del morral para reparar el sector actual y avanzar de nivel
R4	Comprar parche (tienda)	Gastar bits para adquirir el Parche_Ultra y añadirlo al morral
R5	Recargar energía	Intercambiar 15 de integridad por 40 de energía (con validación de integridad mínima)
R6	Combatir jefe final	Usar ítem del morral para dañar el escudo del jefe; ganar si escudo ≤ 0
R7	Finalizar juego	Detectar victoria o derrota y cerrar la aplicación con el mensaje correspondiente
R8	Interfaz gráfica (GUI)	Renderizar el estado del jugador, barras de estadísticas, consola de log y botones de acción
R9	Manejo de excepciones	Capturar todas las GameException y mostrarlas al jugador sin romper la ejecución

3. Especificación y descomposición de requisitos:

3.1. Requisito 1: Iniciar juego

R1	Iniciar juego	Inicializar todos los componentes del juego y lanzar la GUI
Entradas	Ninguna — valores predefinidos en el código.	
Resultados	Se crea Player('Admin_User') con integridad=100, energía=50, bits=0 y morral vacío. Se inicializa la lista de sectores: [Sector('Kernel_Core', 25), Sector('Firewall_Node', 50)]. Se crea el jefe final Boss('GIGA VIRUS', escudo=100). Se crea el ítem de tienda Item('Parche_Ultra', potencia=70, precio=35). Se lanza la ventana tkinter (700×600) con título 'Core-Dump: Advanced System Recovery'. Se muestra el panel de diagnóstico en tiempo real y el log de consola con mensajes de arranque.	
Excepción lanzada	Ninguna en esta fase.	

Descomposición:

Paso	Método	Responsable
Crear instancia del juego	Engine.__init__()	Engine
Crear jugador	Player.__init__('Admin_User')	Player
Crear sectores	Sector('Kernel_Core', 25), Sector('Firewall_Node', 50)	Engine
Crear jefe	Boss('GIGA VIRUS', escudo=100)	Engine
Crear ítem de tienda	Item('Parche_Ultra', potencia=70, precio=35)	Engine
Lanzar interfaz gráfica	CoreDumpGUI.__init__(root)	main.py

Actualizar pantalla inicial	actualizar_pantalla()	CoreDumpGUI
Mostrar reporte jugador	obtener_reporte() → str	Player

3.2. Requisito 2: Explorar sistema

R2	Explorar sistema	El jugador explora consumiendo energía y recibiendo bits o daño aleatoriamente
Entradas	Clic en el botón 'EXPLORAR RED (-5 En)'	
Resultados	Se descuenta 5 unidades de energía del jugador. Con probabilidad > 0.4: se obtienen entre 20 y 40 bits de forma aleatoria. Con probabilidad ≤ 0.4: se resta 10 de integridad al jugador. El resultado se muestra en el log de consola y se actualiza la pantalla.	
Excepción lanzada	InsufficientEnergyError — si energía < 5 antes de explorar.	

Descomposición:

Paso	Método	Responsable
Disparar acción en GUI	ejecutar_explorar()	CoreDumpGUI
Validar energía mínima	if jugador.energia < 5: raise InsufficientEnergyError	Engine
Descontar energía	jugador.energia -= 5	Engine / Player
Generar evento aleatorio	random.random() > 0.4	Engine
Agregar bits (éxito)	jugador.bits += random.randint(20, 40)	Engine / Player
Aplicar daño (fallo)	jugador.integridad -= 10	Engine / Player
Mostrar resultado en consola	log_consola(resultado)	CoreDumpGUI
Refrescar estadísticas GUI	actualizar_pantalla()	CoreDumpGUI

3.3. Requisito 3: Reparar sector

R3	Reparar sector	Usar un ítem del morral para reparar el sector actual y avanzar de nivel
Entradas	Clic en el botón 'REPARAR SECTOR'	
Resultados	Se verifica que queden sectores pendientes por reparar. Se verifica que el morral del jugador no esté vacío. Se extrae el primer ítem del morral con morral.pop(0). Si potencia del ítem > dificultad del sector: nivel_actual += 1 y se informa éxito. Si potencia ≤ dificultad: se informa que el parche no es suficiente (ítem consumido igualmente).	
Excepciones lanzadas	AllSectorsRepairedError — si nivel_actual >= len(sectores). EmptyInventoryError — si morral está vacío.	

Descomposición:

Paso	Método	Responsable
Disparar acción en GUI	ejecutar_reparar()	CoreDumpGUI
Verificar sectores pendientes	if nivel_actual >= len(sectores): raise AllSectorsRepairedError	Engine
Verificar morral no vacío	if not jugador.morral: raise EmptyInventoryError	Engine
Extraer ítem del morral	parche = jugador.morral.pop(0)	Engine / Player
Obtener sector actual	sectores[nivel_actual]	Engine
Comparar potencia vs dificultad	if parche.potencia > sector.dificultad	Engine
Avanzar de nivel (éxito)	nivel_actual += 1	Engine
Refrescar GUI	actualizar_pantalla()	CoreDumpGUI

3.4. Requisito 4: Comprar parche (Tienda)

R4	Comprar parche	Adquirir el Parche_Ultra gastando bits
Entradas	Clic en el botón 'COMPRAR PARCHE (35 Bits)'	
Resultados	Se verifica que el jugador tenga bits suficientes (≥ 35). Se descuentan 35 bits al jugador. Se añade una instancia de parche_pro (Parche_Ultra, potencia=70) al morral. Se muestra el mensaje de compra exitosa en la consola.	
Excepción lanzada	InsufficientBitsError — si bits < precio del parche.	

Descomposición:

Paso	Método	Responsable
Disparar acción en GUI	ejecutar_tienda()	CoreDumpGUI
Verificar bits suficientes	if jugador.bits < parche_pro.precio: raise InsufficientBitsError	Engine
Descontar bits	jugador.bits -= parche_pro.precio	Engine / Player
Añadir ítem al morral	jugador.morral.append(parche_pro)	Engine / Player
Notificar resultado	log_consola(resultado)	CoreDumpGUI
Refrescar GUI	actualizar_pantalla()	CoreDumpGUI

3.5. Requisito 5: Recargar energía

R5	Recargar energía	Intercambiar integridad por energía con validación de integridad mínima
Entradas	Clic en el botón 'INYECTAR ENERGÍA (-15 Hp)'	
Resultados	Se verifica que la integridad sea mayor a 15 (margen de seguridad). Se descuentan 15 unidades de integridad. Se añaden 40 unidades de energía (con tope máximo de 100). Se muestra el mensaje de recarga en consola.	
Excepción lanzada	LowIntegrityError — si integridad $\leq$ 15 antes de recargar.	

Descomposición:

Paso	Método	Responsable
Disparar acción en GUI	ejecutar_recargar()	CoreDumpGUI
Validar integridad mínima	if integridad $\leq$ 15: raise LowIntegrityError	Engine
Descontar integridad	jugador.integridad -= 15	Engine / Player
Añadir energía	jugador.energia += 40	Engine / Player
Aplicar tope de energía	if energia > 100: energia = 100	Engine
Notificar resultado	log_consola(resultado)	CoreDumpGUI
Refrescar GUI	actualizar_pantalla()	CoreDumpGUI

3.6. Requisito 6: Combatir jefe final

R6	Combatir jefe	Usar un ítem del morral para reducir el escudo del GIGA VIRUS
Entradas	Clic en el botón 'PURGAR ENEMIGO (JEFE)'	
Resultados	Si el morral está vacío: el jugador recibe 20 de daño directo a la integridad. Si hay ítem disponible: se usa el primero del morral para dañar el escudo del jefe. Si el escudo del jefe llega a 0 o menos: ejecutando = False y se muestra pantalla de victoria. Si el escudo sigue positivo: se muestra el escudo restante en la consola.	
Excepción lanzada	EmptyInventoryError — si morral vacío (daño de 20 aplicado antes de lanzar la excepción).	

Descomposición:

Paso	Método	Responsable
Disparar acción en GUI	ejecutar_jefe()	CoreDumpGUI
Verificar morral	if not morral: integridad -= 20; raise EmptyInventoryError	Engine
Extraer ítem del morral	ataque = jugador.morral.pop(0)	Engine / Player
Aplicar daño al jefe	jefe_final.recibir_danio(ataque.potencia)	Boss
Verificar victoria	if jefe_final.escudo $\leq$ 0	Engine
Establecer fin de juego	ejecutando = False	Engine

Mostrar diálogo de victoria	messagebox.showinfo('VICTORIA DEL SISTEMA')	CoreDumpGUI
Refrescar GUI	actualizar_pantalla()	CoreDumpGUI

3.7. Requisito 7: Finalizar juego

R7	Finalizar juego	Detectar condiciones de victoria o derrota y terminar la sesión
Entradas	Condición automática: integridad ≤ 0 (derrota) o escudo del jefe ≤ 0 (victoria).	
Resultados	Derrota: la pantalla parpadea en rojo y se muestra messagebox.showerror con 'CRITICAL COLLAPSE'. Victoria: messagebox.showinfo con 'VICTORIA DEL SISTEMA' y 'GIGA VIRUS purgado exitosamente'. En ambos casos se llama a root.quit() para cerrar la aplicación.	
Excepción lanzada	Ninguna — la lógica se evalúa en actualizar_pantalla().	

Descomposición:

Paso	Método / Lógica	Responsable
Verificar estado del jugador	jugador.esta_vivo() $\rightarrow$ bool	Player / Actor
Derrota: parpadeo de pantalla	flash_pantalla('#ff0000')	CoreDumpGUI
Derrota: mostrar mensaje	messagebox.showerror('CRITICAL COLLAPSE')	CoreDumpGUI
Victoria: mostrar mensaje	messagebox.showinfo('VICTORIA DEL SISTEMA')	CoreDumpGUI
Cerrar aplicación	root.quit()	CoreDumpGUI

3.8. Requisito 8: Interfaz gráfica (GUI)

R8	Interfaz gráfica	Renderizar el estado del sistema en tiempo real con Tkinter
Entradas	Cada acción del usuario actualiza la GUI automáticamente.	
Resultados	Panel de diagnóstico con nombre del jugador, bits e inventario. Barra de vida (integridad): verde si $> 40\%$, roja si $\leq 40\%$. Barra de energía en azul cian. Etiqueta del sector actual con nombre y dificultad de cifrado. Etiqueta del estado del jefe con escudo actualizado. Consola de texto con log de eventos (desplazable, fondo oscuro). 5 botones de acción con colores diferenciados y efectos hover.	
Excepción lanzada	Ninguna directamente — todas son capturadas por los métodos ejecutar_*. Se muestran via messagebox.showwarning().	

Descomposición:

Paso	Método / Lógica	Responsable
Inicializar ventana principal	tk.Tk() + root.geometry('700x600')	CoreDumpGUI.__init__
Crear panel de diagnóstico	tk.LabelFrame(text='DIAGNÓSTICO...')	CoreDumpGUI.__init__
Crear barras de estadísticas	dibujar_barra(canvas, valor, max, color)	CoreDumpGUI
Crear consola de log	tk.Text(state='disabled')	CoreDumpGUI.__init__
Crear botones de acción	crear_botones_estilizados()	CoreDumpGUI
Actualizar estado en pantalla	actualizar_pantalla()	CoreDumpGUI
Registrar evento en consola	log_consola(texto: str)	CoreDumpGUI
Parpadeo visual de alerta	flash_pantalla(color_flash: str)	CoreDumpGUI

3.9. Requisito 9: Manejo de excepciones

R9	Manejo de excepciones	Capturar todas las GameException y notificar al jugador sin interrumpir la ejecución
Entradas	Cualquier acción del jugador que viole una regla de negocio.	
Resultados	Se captura la excepción en el bloque except GameException correspondiente. Se muestra un messagebox.showwarning con el título del error y el mensaje de la excepción. El bloque finally garantiza que actualizar_pantalla() siempre se ejecute. La ejecución del juego continúa sin interrupciones.	
Excepciones manejadas	InsufficientEnergyError, InsufficientBitsError, EmptyInventoryError, LowIntegrityError, AllSectorsRepairedError.	

Descomposición:

Paso	Método / Lógica	Responsable
Bloque try en cada acción GUI	try: resultado = self.game.accion()	CoreDumpGUI
Captura de excepción	except GameException as e:	CoreDumpGUI
Mostrar advertencia al usuario	messagebox.showwarning(título, str(e))	CoreDumpGUI
Parpadeo de error	flash_pantalla('#441111')	CoreDumpGUI
Garantizar actualización	finally: actualizar_pantalla()	CoreDumpGUI

4. Ultima versión del diseño:

